University of Glasgow

WORLD CHANGING GLASGOW

Lecture 6
Embedded System Project Development Cycle

UESTCHN2004: Embedded Processors

Dr Lina Mohjazi

Senio Lecturer (Associate Professor)

James Watt School of Engineering

University of Glasgow

University of Glasgow 2013 Glasgow College, UESTC

Lesson Plan

Aim

- Explain Embedded System Software Development, Testing, and Debugging

Learning Objectives

By the end of this lesson you will be able to:

- Understand the quality attributes of embedded systems
- Design and development of embedded systems software
- Identify the debugging tools used for embedded systems

Assumed Previous Knowledge

- understanding of the instruction set of ARM microcontroller,
- understanding of assembly language basics
- C programming.

Outline of Lecture 6

- Project Development Cycle Overview
- Embedded System Development Examples
- Firmware

Outline of Lecture 6

- Project Development Cycle Overview
- Embedded System Development Examples
- Firmware

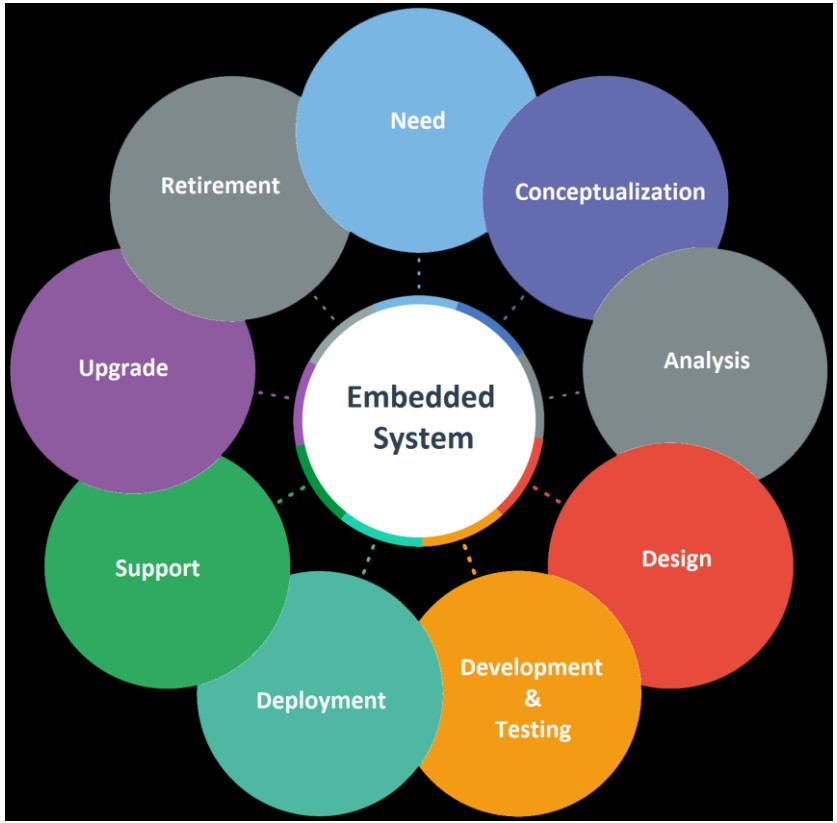
Project Development Cycle

What

Why

How

Embedded Systems Life Cycle



It is an Analysis – Design – Implementation based problem solving approach for embedded systems development.

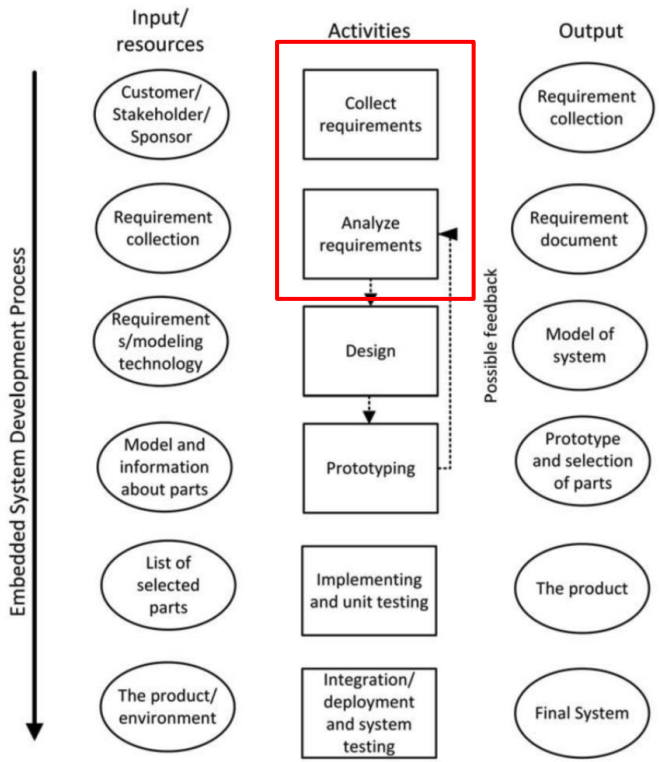
Embedded Systems Life Cycle

Analysis involves understanding what product needs to be developed

Design involves what approach to be used to build the product

Implementation is developing the product by realizing the design.

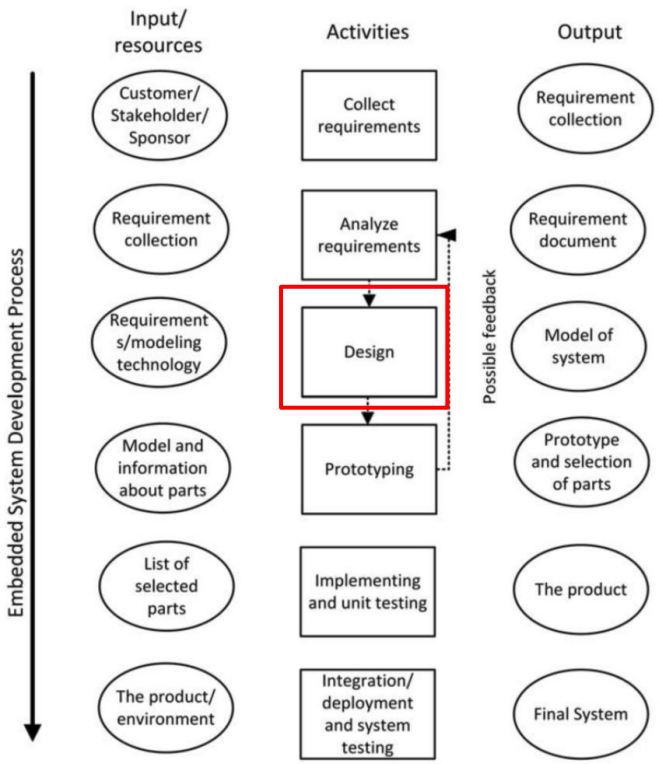
Embedded Systems Life Cycle



Requirement Analysis and Specification:

- Identify and understand the requirements of the embedded system.
- Gather input from stakeholders to determine functionality, performance, constraints, etc.
- Define system specifications including hardware, software, and interface requirements.
- Consider factors such as power consumption, size constraints, and environmental conditions.

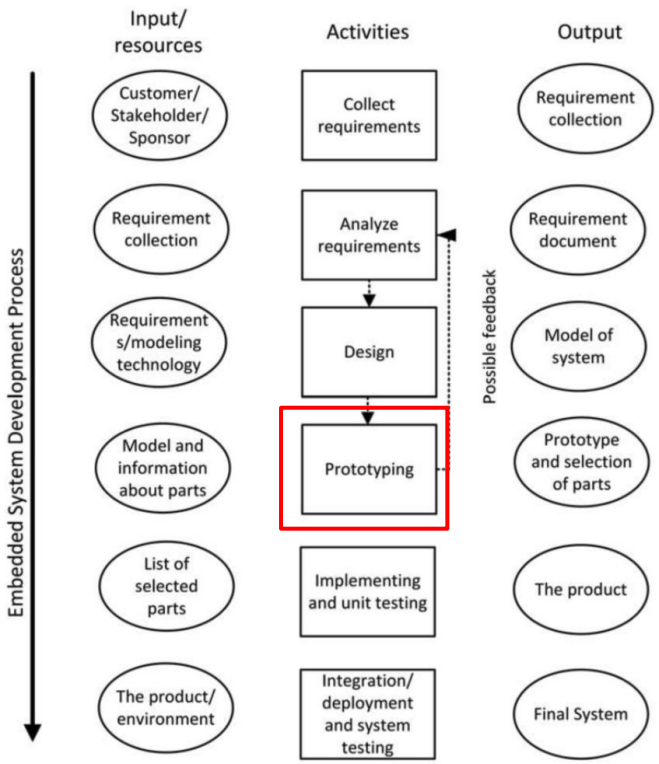
Embedded Systems Life Cycle



System Design:

- Architectural design: Determine the overall system architecture, including the choice of microcontroller/microprocessor, sensors, actuators, communication interfaces, etc.
- Detailed design: Specify individual modules, interfaces, algorithms, and data structures.
- Consider design trade-offs such as performance vs. power consumption, real-time requirements, and cost.

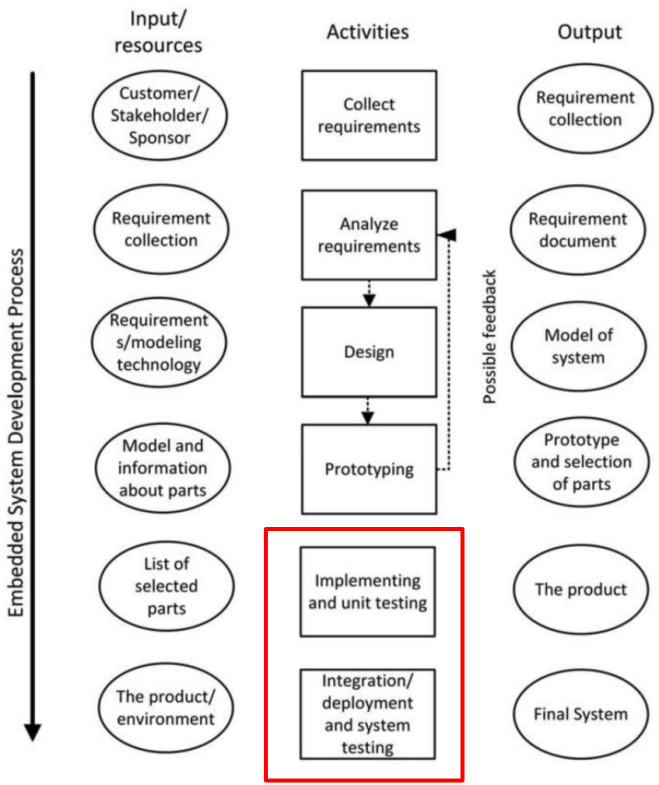
Embedded Systems Life Cycle



Prototyping:

- Develop the hardware components: Design and prototype the PCB (Printed Circuit Board), select components, and perform soldering if necessary.
- Develop the software components: Write firmware/software for the microcontroller/microprocessor, implement algorithms, drivers, and interfaces.
- Integrate hardware and software components.

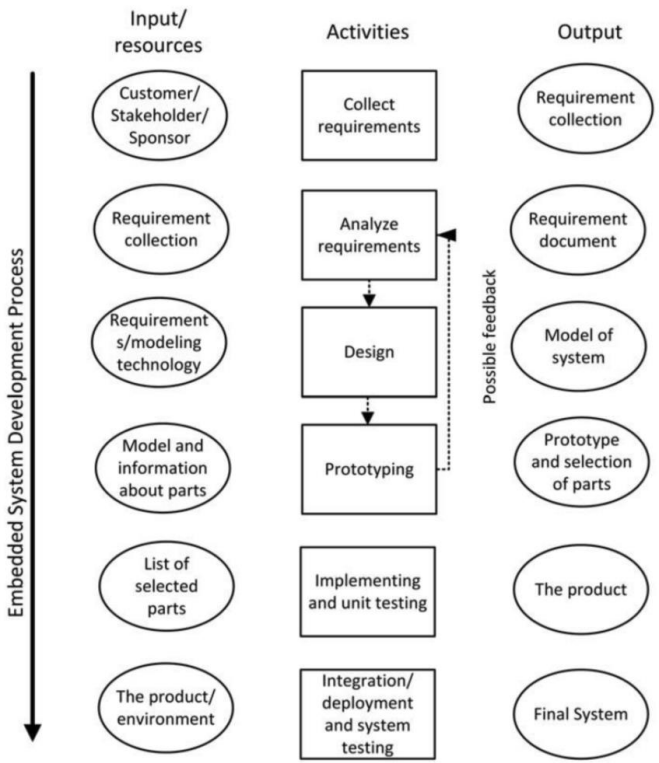
Embedded Systems Life Cycle



Testing and Verification:

- Unit testing: Test individual modules/components to ensure they function correctly.
- Integration testing: Test the interaction between hardware and software components.
- Validation testing: Verify that the system meets the specified requirements and behaves as expected.
- Perform both functional and non-functional testing (e.g., performance, reliability, safety).

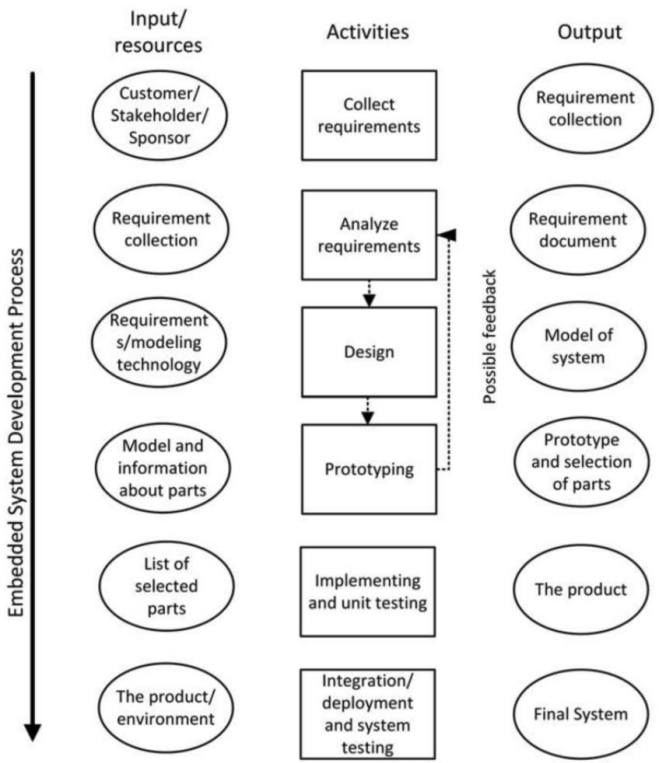
Embedded Systems Life Cycle



Validation and Verification:

- Ensure that the system meets the defined requirements.
- Validate the system against real-world scenarios and user needs.
- Verify compliance with industry standards and regulations.

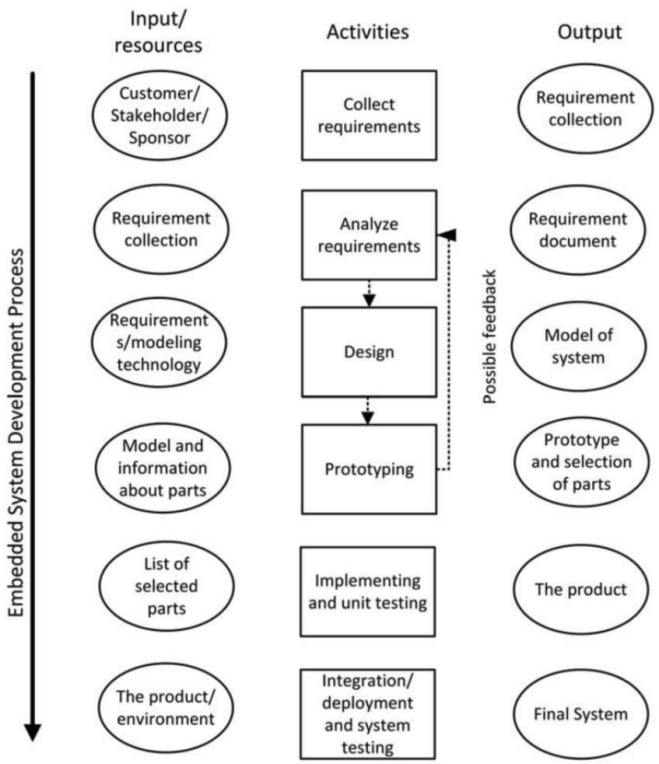
Embedded Systems Life Cycle



Deployment and Maintenance:

- Deploy the embedded system in its intended environment.
- Provide user documentation and training if necessary.
- Establish procedures for system maintenance, including software updates, bug fixes, and hardware upgrades.
- Monitor the system's performance and address any issues that arise during operation.

Embedded Systems Life Cycle



Feedback and Iteration:

- Gather feedback from users and stakeholders.
- Identify areas for improvement or additional features.
- Iterate on the design and implementation based on feedback to enhance the system's functionality, performance, and usability.

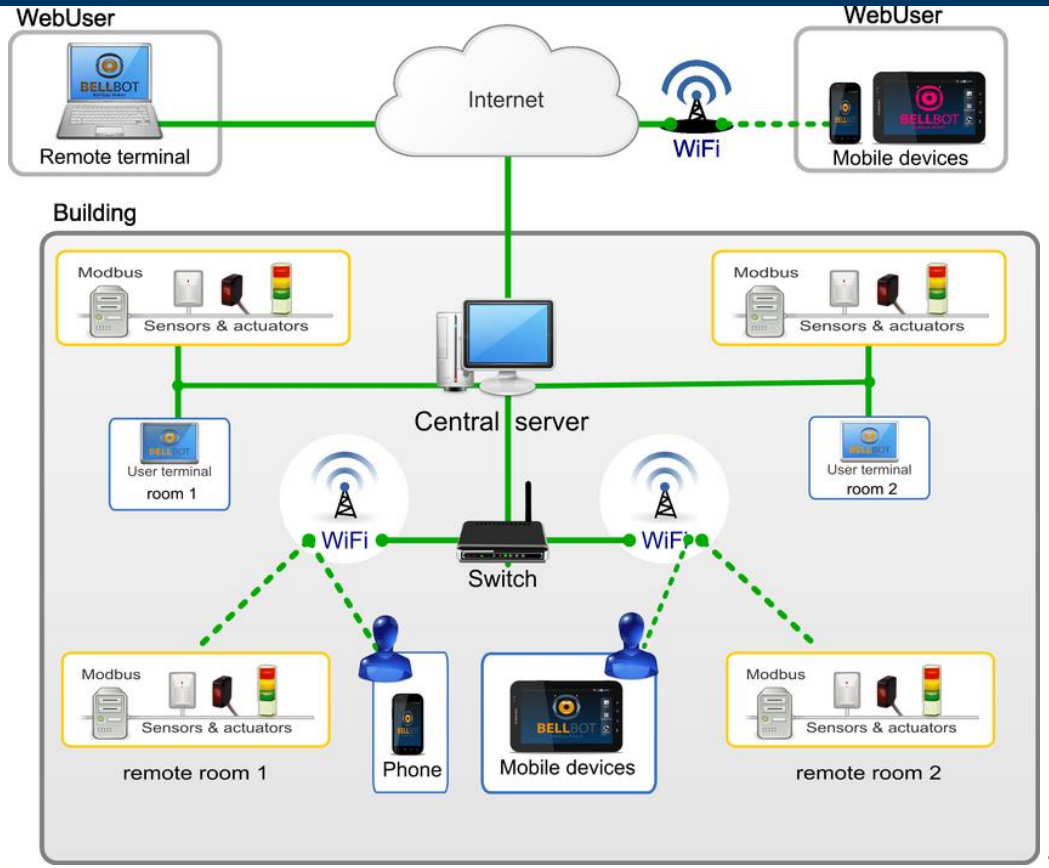
Outline of Lecture 6

- Project Development Cycle Overview
- Embedded System Development Examples
- Firmware

Embedded Systems Life Cycle

Example 1: Home Automation System (Smart Home)

A home automation system refers to a system to monitor and/or control home attributes such as lighting, climate, entertainment systems, and appliances.



Example 1: Home Automation System (Smart Home)

Requirement Analysis?

Identify the need for controlling lights, appliances, security systems remotely.

System Design?

Architectural design: Determine the central control unit, communication protocols (Wi-Fi, Bluetooth), and compatible devices.

Prototyping?

Develop hardware components such as microcontroller-based control units, sensors, actuators.
Write firmware/software for control logic and user interfaces.

Example 1: Home Automation System (Smart Home)

Testing and Verification?

Test individual components (sensors, actuators) and their integration.
Verify system functionality under different scenarios (e.g., turning lights on/off, detecting intrusions).

Deployment and Maintenance?

Deploy the system in a home environment.
Provide user manuals and support for maintenance and updates.

Feedback and Iteration?

Gather feedback from users on system performance and usability.Iterate on the design based on user feedback to enhance functionality or add new features (e.g., voice control).

Example 2: Automated Parking System

Automated Parking System is a mechanical system designed to minimize the area and/or volume required for parking cars.

Example 2: Automated Parking System

Requirement Analysis?

Determine the need for an automated parking system to optimize space utilization and improve efficiency.

Hardware: Microcontroller or embedded computer for system control, sensors (e.g., ultrasonic or infrared sensors) to detect vehicle presence and occupancy, actuators (e.g., motors or hydraulic systems) to move vehicles into designated parking spaces.

Software: Firmware to control sensors and actuators, algorithms for vehicle detection, parking space assignment, and vehicle movement.

Example 2: Automated Parking System

System Design?

Architectural Design:

- Select microcontroller/embedded computer based on processing power, memory, and required interfaces (e.g., GPIO, serial communication).
- Design communication protocols for sensor data acquisition and actuator control.

Detailed Design:

- Define sensor placement and wiring for optimal vehicle detection.
- Develop algorithms for efficient parking space assignment considering factors like vehicle size and available space.

Example 2: Automated Parking System

Prototyping?

Hardware Development:

- Prototype circuitry for sensor interfacing and actuator control.
- Select and integrate motors or hydraulic systems for vehicle movement.

Software Development:

- Write firmware to interface with sensors and actuators, including interrupt-driven routines for real-time responsiveness.
- Implement algorithms for vehicle detection and parking space assignment, ensuring efficient memory and CPU usage.

Example 2: Automated Parking System

Testing and Verification?

Unit Testing:

- Test sensor accuracy and reliability under various lighting and weather conditions.
- Verify actuator functionality and precision in moving vehicles.

Integration Testing:

- Test the complete system with simulated vehicle entry and exit scenarios.
- Verify communication between sensors, actuators, and the control unit.

Validation Testing:

- Validate system performance under real-world conditions, including peak traffic times and varying vehicle sizes

Example 2: Automated Parking System

Validation and Verification?

Validation:

- Deploy the automated parking system in a parking garage environment.
- Monitor system performance and reliability over an extended period.

Verification:

- Ensure the system meets safety standards and regulatory requirements for automated machinery.

Example 2: Automated Parking System

Deployment and Maintenance?

Deployment:

- Install the automated parking system in the parking garage, including sensor calibration and system configuration.
- Provide training for parking attendants and users on system operation and emergency procedures.

Maintenance:

- Establish a schedule for routine maintenance, including sensor calibration and software updates.
- Provide technical support for troubleshooting and resolving issues encountered during operation.

Example 2: Automated Parking System

Feedback and Iteration?

Feedback Collection:

- Gather feedback from parking garage management and users on system performance, reliability, and user experience.

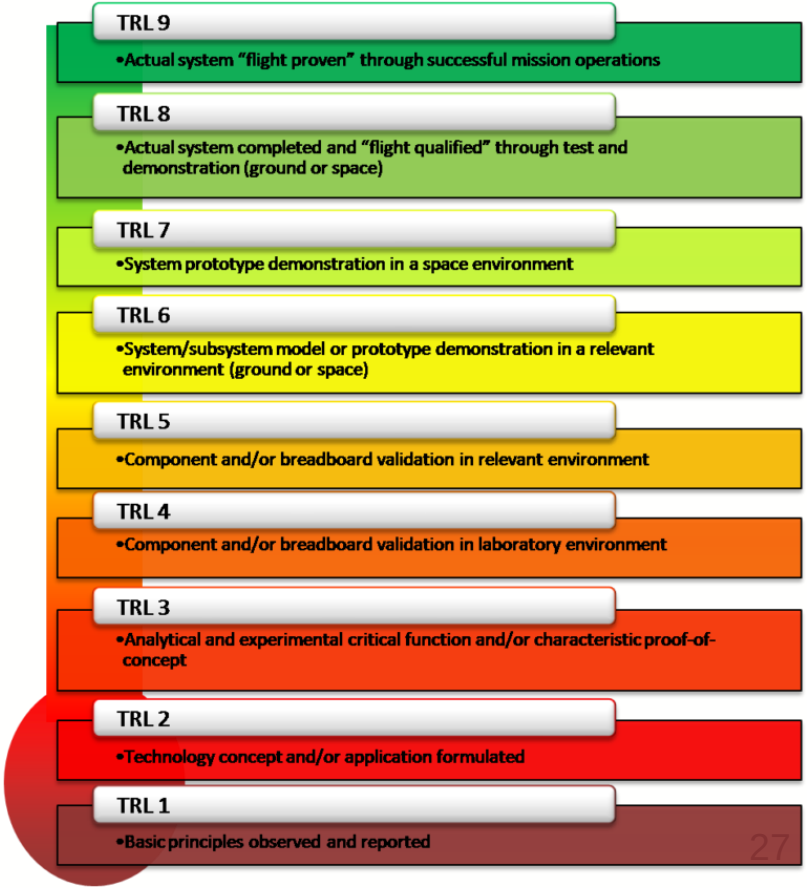
Iteration:

- Based on feedback, make improvements to the system firmware or hardware to address any issues identified.
- Consider adding features such as license plate recognition or mobile app integration to enhance user convenience and security.

Embedded Systems Life Cycle

Technology readiness levels (**TRLs**) are a method for estimating the maturity of technologies.

From the conceptual stage to commercialisation



Quality Attributes in Embedded Systems

Quality Attributes of Embedded System:

Quality attributes are the non-functional requirements that need to be documented properly in any system design

- Quality attributes can be classified as:

1. Operational quality attributes
2. Non-operational quality attributes

- *Operational Quality Attributes:* The operational quality attributes represent the relevant quality attributes related to the embedded system when it is in the operational mode or online mode.

- *Non-Operational Quality Attributes*: The quality attributes that needs to be addressed for the product not on the basis of operational aspects are grouped under this category.

Quality Attributes in Embedded Systems

Outline of Lecture 6

- Project Development Cycle Overview
- Embedded System Development Examples
- Firmware

Embedded Firmware

Introduction:

- **Definition:** The control algorithm (Program instructions) and or the configuration settings that an embedded system developer dumps into the code (Program) memory of the embedded system.
- It is an un-avoidable part of an embedded system.
- The embedded firmware can be developed in various methods like:
  - Write the program in high level languages like **Embedded C/C++** using an Integrated Development Environment (IDE):
    - *The IDE will contain an editor, compiler, linker, debugger, simulator etc.*
    - *IDEs are different for different family of processors/controllers*.
  - Write the program in **Assembly Language** using the instructions supported by your application's target processor/controller

Embedded Firmware – Design and Development

- The embedded firmware is responsible for controlling the various peripherals of the embedded hardware and generating response in accordance with the functional requirements of the product.
- The embedded firmware is the master brain of the embedded system.
- The embedded firmware imparts intelligence to an Embedded system.
- It is a one-time process and it can happen at any stage.
- The product starts functioning properly once the intelligence introduced to the product by embedding the firmware in the hardware.
- The product will continue serving the assigned task till hardware breakdown occurs or a corruption in embedded firmware.
- In case of hardware breakdown, the damaged component may need to be replaced and for firmware corruptions the firmware should be re-loaded, to bring back the embedded product to the normal functioning.

Embedded Firmware – Design and Development

- The embedded firmware is usually stored in a permanent memory (ROM) and it is non-alterable by end users.

- Designing Embedded firmware requires understanding of:

  - embedded product **hardware**:
    - interfacing components
    - memory map details
    - I/O port details
    - configuration and register details of various hardware chips used.

  - some programming language
    - Low level Assembly Language
    - High level language like C/C++
    - a combination of the two.

Embedded Firmware – Design and Development

- There exist two basic approaches for the design and implementation of embedded firmware, namely:

  1. The Super loop based approach

  2. The Embedded Operating System based approach

- **The decision on which approach needs to be adopted for firmware development is purely dependent on the <u>complexity</u> and <u>system requirements</u>**

Embedded Firmware – Super Loop Approach

- **The Super loop:** The Super loop based firmware development approach is suitable for applications that are *not time critical* and where the *response time is not so important* (Ex. Embedded systems where missing deadlines are acceptable).
  - It is very similar to a conventional procedural programming where the code is executed task by task.
  - The tasks are executed in a never ending loop.
  - The task listed on top on the program code is executed first and the tasks just below the top are executed after completing the first task.

Embedded Firmware – Super Loop Approach

- A typical super loop implementation will look like:
  1. Configure the common parameters and perform initialization for various hardware components memory, registers etc.
  2. Start the first task and execute it
  3. Execute the second task
  4. Execute the next task
  5. :
  6. :
  7. Execute the last defined task
  8. Jump back to the first task and follow the same flow.

The 'C' program code for the super loop is given below

```
void main ()
{
Configurations ();
Initializations ();
while (1)
{
Task 1 ();
Task 2 ();
:
:
Task n ();
}
}
```

Embedded Firmware – Super Loop Approach

- **Pros:**
  - Doesn't require an Operating System for task scheduling and monitoring and free from OS related overheads.
  - Simple and straight forward design.
  - Reduced memory footprint.

- **Cons:**
  - Non-Real time in execution behaviour (*As the number of tasks increases the frequency at which a task gets CPU time for execution also increases*).
  - Any issues in any task execution may affect the functioning of the product (This can be effectively tackled by using Watch Dog Timers for task execution monitoring).

- **Enhancements:**
  - Combine Super loop based technique with interrupts.
  - Execute the tasks (like keyboard handling) which require Real time attention as Interrupt Service routines. (An example will be given in the next lecture)

Embedded Firmware – Embedded OS based Approach

- **Embedded OS based Approach**: The embedded device contains an Embedded Operating System which can be one of:

1. A Real Time Operating System (RTOS).
2. A Customized General-Purpose Operating System (GPOS)

- Categorized based on: **1) type of kernel and kernel services, 2) purpose and type of computing systems where the OS is deployed, 3) the responsiveness to applications.**

- The Embedded OS is responsible for **scheduling the execution** of user tasks and the **allocation of system resources** among multiple tasks.

- It Involves high OS related overheads apart from managing and executing user defined tasks.

- Point of Sale (PoS) terminals, Gaming Stations, Tablet PCs etc are examples of embedded devices running on embedded **GPOS**s.

- 'Mobile Phones, Flight Control Systems etc are examples of embedded devices that runs on **RTOS**s

Embedded Firmware – RTOS vs. GPOS

RTOS

- Unfair scheduling
  - Scheduling based on priority
- Kernel is preemptive either completely or up to maximum degree
- Priority inversion is a major issue
- Predictable behavior

GPOS

- Fair scheduling
  - Scheduling can be adjusted dynamically for optimized throughput
- Kernel is non-preemptive or have long non-preemptive code sections
- Priority inversion usually remain unnoticed
- No predictability guarantees

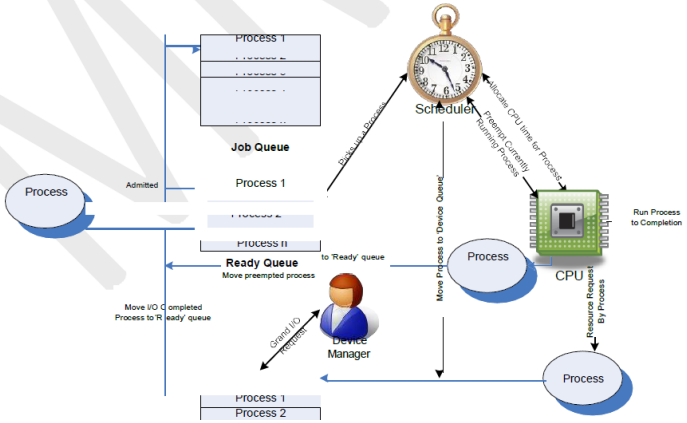
Task Scheduling

- **The task scheduler**
  - the part of the operating system that **responds** to the requests by programs and interrupts for processor attention and **gives** control of the processor to those processes.

- **Scheduling algorithm** : The algorithm followed to decide **who gets next turn on CPU**.

- The program that does this is called the **Scheduler**

- **In real-time systems,**
  - every real-time task has a **deadline** before or at which it must be completed.
  - the scheduling algorithms of these systems must be timely and predictable, Due to the criticality of the tasks,
- **Task scheduling:**
  - Tasks could either be **periodic** or **aperiodic**.
  - Real-time scheduling algorithms may assign priorities **statically**, **dynamically**, or in a **hybrid manner**, which are called fixed, dynamic and mixed scheduling algorithms, respectively.
  - These algorithms may **allow preemptions** to occur or may impose a **non-preemptive method**.



Task Scheduling - Scheduler Selection:

- CPU Utilisation
- Throughput
- Turnaround Time
- Waiting Time
- Response Time

To summarise, a good scheduling algorithm has high CPU utilisation, minimum Turn Around Time (TAT), maximum throughput and least response time.

Preemptive vs. Nonpreemptive Scheduling

- **Preemptive** processes
  - Can be removed from their current processor
  - Can lead to improved response times
  - Important for interactive environments
  - Preempted processes remain in memory
- **Nonpreemptive** processes
  - Run until completion or until they yield control of a processor
  - Unimportant processes can block important ones indefinitely

15.2.2 Preemptive Scheduling

In a preemptive scheduling once the CPU is given to a task it can be taken away, for example when a higher priority task wants the CPU. Preemptive scheduling is used in real-time systems where the tasks are usually configured with different priorities and time critical tasks are given higher priorities. A higher priority task can stop a lower priority one and grab and use the CPU until it releases it. In preemptive scheduling the task contexts are saved so that the tasks can resume their operations from the point they left when they are given back the CPU.

Embedded Firmware – Time Management

Based on the degree of tolerance in the timing constraints, RTOSs are classified into three types:

Hard, Soft & Firm Real Time Systems:

- **Hard Real Time Systems:** *In Hard real time systems the processing deadlines are missed, the system had said to be failed.*

  (Eg: Missile may not attack the target. Aircraft may crash, Nuclear Power plant may explode).
- **Soft Real Time Systems:** *In soft real time systems must meet the deadline but only in the average case.*

  (Eg: Screen switching at 250 m sec, Maintain and update of flight plans in commercial airlines, Live audio/Video systems)
  - *A soft real-time operating system is one that has reduced constraints on 'lateness,' but still must operate quickly within fairly consistent time constraints."*
- **Firm Real Time Systems:** *These systems that are primarily soft, but do have a hard deadline as well.*

Hard, Soft & Firm Real Time Systems:

- It is important to note that **hard** versus **soft real-time** does not necessarily relate to the length of time available.
  - A machine may overheat if a processor does not turn on cooling within **15 minutes**. Still it is a **Hard real time** system.
  - A network interface card may lose buffered data if it is not read within a **fraction of a second**, but the data can be resent over the network without major adverse consequences. It is a **Soft Real** time system.

University of Glasgow

University of Glasgow 2013 Glasgow College, UESTC

WORLD CHANGING GLASGOW

Summary

- Project Development Cycle Overview
- Embedded System Development Examples
- Firmware

In the Next Lesson:

- Testing and Debugging